PURDUE UNIVERSITY | Rosen Center for Advanced Computing

Negishi Recursive Migration

Ensuring Continuous VM Uptime on HPC Clusters

by Samuel Gomez

EXECUTIVE SUMMARY

High-performance computing (HPC) environments often impose strict per-job walltime limits that can interrupt long-running virtual machine (VM) workloads. This paper presents a production-tested approach for maintaining continuous VM uptime on Purdue's Negishi HPC cluster by combining QEMU live migration with SLURM job orchestration. The method launches a source VM, monitors walltime across queued/running jobs, and when a job nears its limit, automatically provisions a destination VM on a different node and performs a live, in-memory state transfer. A recurring SLURM scrontab job coordinates these transitions, achieving near-zero guest downtime and enabling multi-hour and multi-day continuous VM lifetimes despite per-job limits.

We document the design, operation, and implementation, including component roles, control flow, and operational procedures. This approach is suitable for HPC operators, Research Solutions Engineers, and researchers who require persistent VM runtimes for interactive analysis, long simulations, services, or iterative development.

Purdue's Rosen Center for Advanced Computing (RCAC)

TABLE OF CONTENTS

INTRODUCTION & BACKGROUND

Modern HPC centers typically prefer batch scheduling to share resources fairly across users and projects. SLURM, an open-source scheduler, enforces resource quotas and walltime limits per job. This is beneficial for throughput and fairness, but introduces friction for stateful or interactive workloads inside VMs that are not naturally checkpoint/restart friendly.

QEMU provides live migration, allowing a running VM's memory and device state to be streamed to another hypervisor instance, minimizing pause time for the guest. In cloud and virtualization platforms, this underpins non-disruptive maintenance and load balancing. However, on shared academic clusters, live migration typically isn't exposed as a managed service.

This project operationalizes QEMU live migration on top of SLURM by scripting the orchestration steps needed to:

1. Detect when a VM's job is approaching its time limit.
2. Launch a fresh destination VM process that listens for an incoming state.
3. Trigger QEMU's migration over the HPC interconnect from the source to the destination.
4. Repeat the process before each new job boundary, recursively sustaining uptime.

PROBLEM DESCRIPTION

Long-running VM workloads on SLURM-managed clusters are interrupted when job walltime expires, causing loss of in-memory state, open network sessions, and long application warm-up costs.

Causes

- HPC jobs policies require finite wall times (e.g. 90 minutes).
- Many VM use cases are stateful and interactive, not trivially restartable.
- Users often lack permission for hypervisor-level features that automate live migration.

Impact

- Disrupted research workflows and services.
- Manual restarts, re-logins, and state re-creations.
- Reduced productivity and unreliable service availability.

CRITERIA FOR ACCEPTABLE SOLUTIONS

In the context of Purdue's Negishi HPC cluster and similar research computing environments, any solution designed to sustain long-running virtual machines must satisfy a set of operational and technical criteria. The following standards define what makes a solution both practical and compliant for continuous VM uptime within HPC constraints.

1. Preserve VM state

The system must carry over the full runtime state of the virtual machine (including memory and CPU) across SLURM job boundaries with minimal pause, ensuring no data loss or restart.

2. Integrate with SLURM

The solution should work through standard SLURM tools such as **sbatch**, **squeue**, and **scontrol**, allowing the scheduler to handle job submission, monitoring, and resource limits normally.

3. Be automated

Once configured, the process should run without user intervention, detecting job time limits, launching new VMs, and triggering live migrations automatically through **scrontab**.

SOLUTION OVERVIEW

The **Negishi Recursive Migration System** provides a fully automated mechanism to sustain VM uptime on Purdue's Negishi HPC cluster. The approach integrates **QEMU live migration** with **SLURM job scheduling** and automated orchestration scripts, enabling continuous VM operation across SLURM job boundaries.

In a typical SLURM-managed environment, compute jobs are terminated when their limits expire, disrupting long-running processes. This solution addresses that limitation by introducing a **recursive migration process** that moves VM's live state from one compute node to another before the time limit is reached. The transition is performed transparently (without restarting the guest operating system) ensuring continuous availability.

How the Recursion Works

The system begins with a **Source VM,** launched as the initial QEMU instance within a SLURM job. As the job approaches its time limit, a **Hybrid VM** is deployed on another node. This hybrid instance is uniquely configured to receive the live migration from the current VM while being capable of initiating the next migration cycle (as the sender). Once the migration is complete, the hybrid VM becomes the new "source", allowing the cycle to continue indefinitely.

The following diagram conceptually illustrates this process:

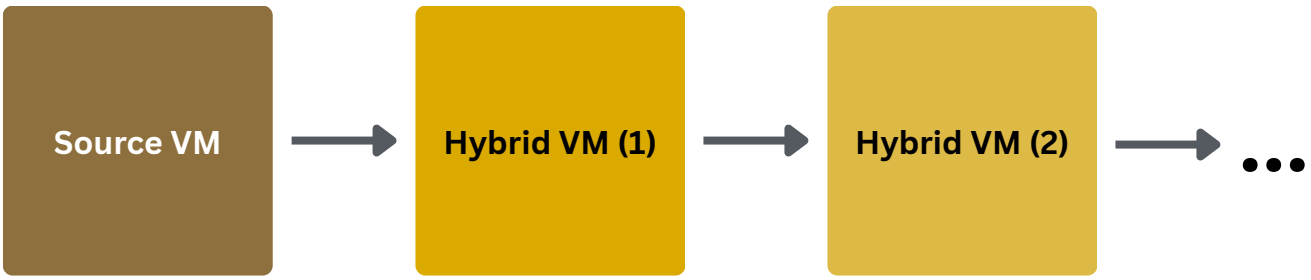


Figure 1: Conceptual Overview of Recursive Migration

The Source VM migrates to Hybrid VM (1), which later migrates to Hybrid VM (2), then Hybrid VM (3), and so on, forming a continuous, self-sustaining sequence.

Each migration cycle consists of three automated phases:

1. **Detection:** The system checks all active SLURM jobs and determines if a migration is necessary based on remaining runtime.
2. **Deployment:** A new destination VM is automatically scheduled using SLURM's **sbatch** command, ensuring it runs on a different node.
3. **Migration:** The current VM transfers its complete runtime state (including memory and CPU context) to the destination VM via **QEMU's Machine Protocol (QMP).**

Once the transfer is complete, the destination becomes the new active VM. This sequence repeats at regular intervals, scheduled through SLURM's **scrontab,** resulting in uninterrupted VM uptime.

SYSTEM COMPONENTS & CONTROL FLOW

The system is composed of four key scripts, each fulfilling a specific role within the automation pipeline. Together, they create a self-regulating control loop that maintains continuous operation.

1. **recursive_migration.sub** - The Orchestrator

This Bash script serves as the central control unit of the system. It is called periodically via **SLURM's scrontab** scheduler and manages the entire VM lifecycle. Its primary responsibilities include:

- Detecting whether an active source or hybrid VM job is currently running.
- Retrieving the remaining runtime of the job using **squeue** and converting it to total seconds.
- Submitting a new destination VM using **SLURM's sbatch** command when less than one hour remains.
- Generating a migration command file via **create_migration_qmp_file.sh**.
- Executing the live migration process using **SSH** and **Netcat** to send commands to the **QEMU Machine Protocol (QMP)** interface.

2. **create_migration_qmp_file.sh** - The Migration Command Generator

This script generates the **JSON**-based command file that QEMU uses to initiate live migration. For a given destination node, it creates a file named **run_migrate_<destination_node>.qmp** containing the following commands:

```
{"execute":"qmp_capabilities"}
{"execute":"migrate","arguments":{"uri":"tcp":
<destination_node>:4444"}}
{"execute":"query-migrate"}
```

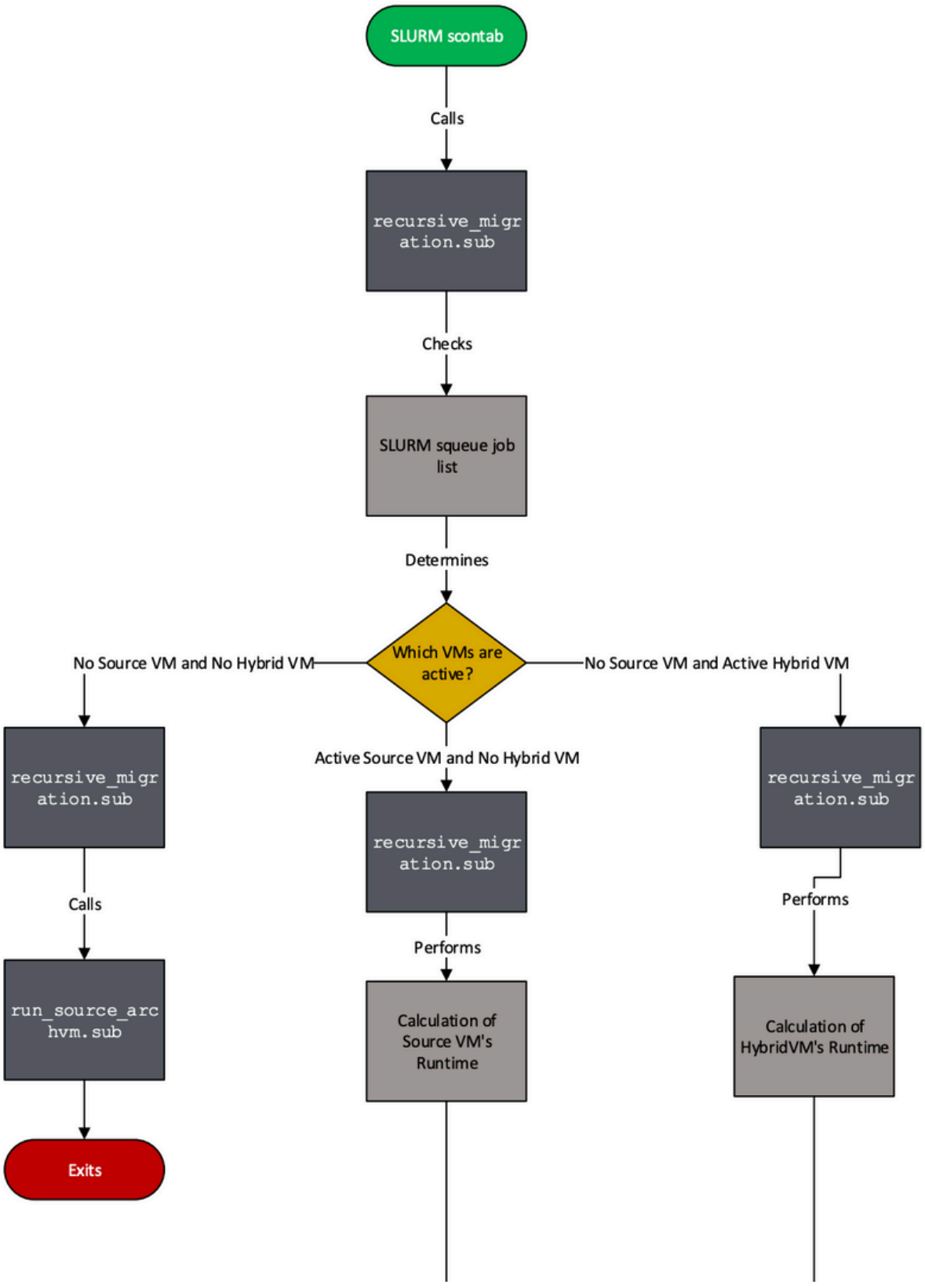
This file is later sent to the QMP interface on the source node, instructing QEMU to begin migration process.

3. **run_source_archvm.sub** - Initial Source VM Job

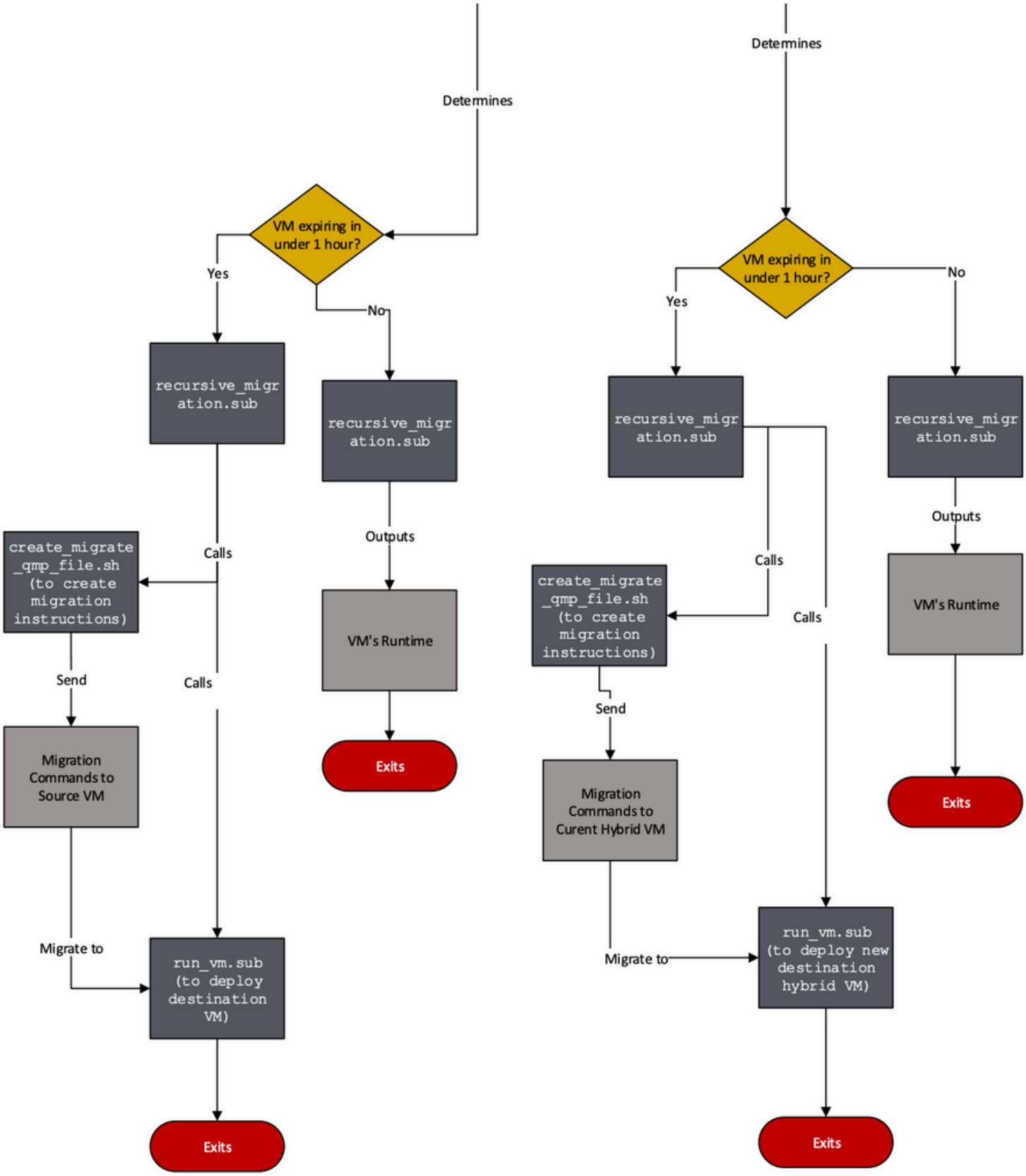
This SLURM job script launches the first QEMU instance (the Source VM) as a foreground process, ensuring that SLURM directly manages its execution lifecycle.

4. **run_vm.sub** - Hybrid VM Job

This job script defines the Hybrid VM configuration, which is capable of both receiving and initiating live migrations. It listens for incoming migration data.



Purdue's Rosen Center for Advanced Computing (RCAC)



Purdue's Rosen Center for Advanced Computing (RCAC)

Figure 2: Control Flow of the Recursive Migration System

A recurring script supervises VM status and transitions the workload to a new instance whenever required.

Control Flow Overview

The system begins with an initial run in which no VMs are active. During this first invocation, the orchestrator detects that no SLURM jobs are running and submits the source VM job. It then exits and waits for the next scheduled execution. On later runs (triggered every 30 minutes by SLURM's scrontab), the orchestrator checks whether a VM is approaching its time limit. If a source VM is still running but has less than one hour remaining, the script launches a new hybrid VM, waits a few minutes for it to initialize, generates the appropriate QMP migration file, and performs the live migration. If the source VM has already been replaced and only hybrid VMs remain, the orchestrator identifies the most recent hybrid instance and carries out the same migration procedure from that VM to a newly launched one.

This cycle continues indefinitely. With scrontab invoking the orchestrator at regular intervals, the VMs seamlessly "hops" from node to node before any job reaches its walltime. Throughout this process, the guest operating system remains active without interruption, achieving continuous uptime with minimal observed downtime during migrations.

CONCLUSION

The recursive migration system developed for Purdue's Negishi cluster demonstrates that long-running VMs can operate reliably within a walltime-restricted HPC environment. By using only SLURM tools and QEMU's live migration capabilities, the solution meets all key criteria established at the outset: it preserves VM state across job boundaries, integrates with the existing scheduler, and runs fully autonomously once deployed.

The orchestrator's ability to detect expiring jobs, provision new hybrid VMs, and transfer system state without interruption ensures continuous uptime of the guest OS. Because the design relies solely on user-level scripts, modules, and permitted network ports, it remains compliant with RCAC policies and does not require system-level modifications or administrator intervention.

The broader implications are significant. This approach allows researchers to maintain persistent development environments, interactive workflows, and stateful services on a cluster designed for batch computation. It also establishes a repeatable pattern for running long-lived virtualized workloads in scheduled compute environments, offering a foundation for future enhancements such as automated health checks, improved scheduling logic, or integration with container-based systems.

In summary, the recursive migration framework provides a practical, policy-aligned method for sustaining VMs indefinitely on HPC infrastructure. It solves a longstanding operational challenge while remaining simple to maintain and adaptable to other clusters or research workflows.

REFERENCES

ArchWiki. (n.d.-a). Arch Boot Process. Arch boot process- ArchWiki. https://wiki.archlinux.org/title/Arch_boot_process#Boot_loader

ArchWiki. (n.d.-b). Installation guide. Installation guide - ArchWiki. https://wiki.archlinux.org/title/Installation_guide

ArchWiki. (n.d.-c). QEMU. QEMU - ArchWiki. https://wiki.archlinux.org/title/QEMU

Learn Linux TV. (n.d.). How To Install Arch Linux The First Time - Complete Walkthrough. YouTube. https://www.youtube.com/watch?v=FxeriGuJKTM

Migration. KVM. (n.d.). https://www.linux-kvm.org/page/Migration

Migration. Migration - QEMU 8.1.5 documentation. (n.d.). https://qemu.readthedocs.io/en/v8.1.5/devel/migration.html

QEMU user Documentation. QEMU User Documentation - QEMU documentation. (n.d.). https://www.qemu.org/docs/master/system/qemu-manpage.html

Slurm Workload manager. Slurm Workload Manager - Documentation. (n.d.). https://slurm.schedmd.com/

Slurm Workload manager. Slurm Workload Manager - scrontab. (n.d.). https://slurm.schedmd.com/scrontab.html

Using QEMU-system-x86_64. Dr Mike Murphy RSS. (n.d.). https://ww2.coastal.edu/mmurphy2/oer/qemu/x86_64/

ACKNOWLEDGEMENTS

This project was made possible through the support, guidance, and expertise of many individuals within Purdue RCAC. I would like to extend my appreciation to my manager, Alex Younts, whose mentorship, technical insight, and continuous encouragement were instrumental throughout the development of this work. His direction helped shape both the engineering approach and the overall vision of the project.

Thank you for your support and dedication in helping bring this project to completion.

Contact

Rosen Center for Advanced Computing
3314
Convergence
101 Foundry Dr
West Lafayette, IN 47906
rcac-help@purdue.edu
+1 765-49-43500